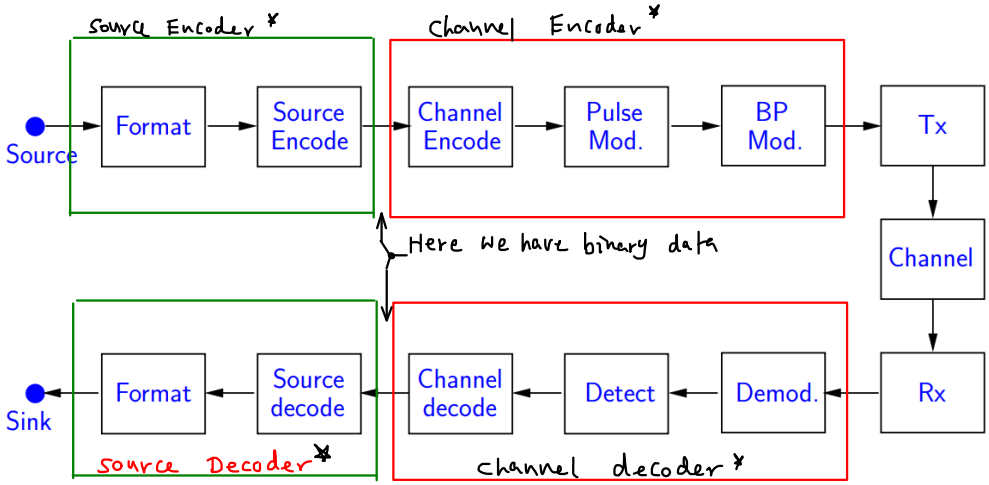
Let us recall the digital communication system, a holistic view



* In different books different terminologies are used. But essentially the same idea. Thus, we use the following terminologies

Format + Source Encode blocks ≡ SOURCE ENCODER

Channel Encode + Pulse Modulate + Band pass Modulate Blocks ≡ CHANNEL ENCODER

SOURCE DECODER and CHANNEL DECODER are defined as shown in the figure above

Generally speaking :

↳ SOURCE ENCODER does the following

→ If "Source" is an analog information source [e.g., voice signal $x(t)$]
↳ signal is sampled, then quantized, then quantized values are mapped with binary words. binary words can be either of fixed length, or of variable length. The binary words constitute a binary stream of data which is sent to the CHANNEL ENCODER

→ If "Source" is an textual information source [e.g., Key board]
↳ the finite number of text symbols are mapped with binary words. binary words can be either of fixed length, or of variable length. The binary words constitute a binary stream of data which is sent to the CHANNEL ENCODER

→ If "Source" is an digital information source
↳ we have already got a binary stream of data. which is directly sent to the CHANNEL ENCODER

Generally speaking :

↳ CHANNEL ENCODER → does the following

① Take as input the sequence of bits [binary stream of data].

② Map the sequence of bits into a sequence of real or complex numbers

③ Use the afore mentioned sequence of real/complex numbers to constitute a WAVEFORM which will be fed to the transmitter [Tx]

Example 1: Consider a keyboard with just 4 characters A, B, C, D. A person is typing a book at a rate $\frac{1}{2}$ characters per second. These stream of characters are fed to Source Encoder. Then to Channel Encoder and finally to the Tx. [assume real-time data transmission]

- Let us start by discussing the functionality of the SOURCE ENCODER
  - At the source encoder : the following takes place
    - This is textual information
    - there are 4 text symbols. Use the following mapping

text symbol	binary word [Fixed length]
A	00
B	01
C	10
D	11

→ time↑

- Suppose the character stream for first 10 seconds is "ABCAA"
- The corresponding source encoder output is 00 | 01 | 10 | 00 | 00 → time↑
- The source encoder output bit rate is 1 bits/sec
- Note that the above mapping given in the table is a "fixed length" mapping
- One could use a variable mapping as well [There are ways of doing it ensuring unique decodability at the reciever Rx]

↓ an example is given below !

text symbol	binary word [Fixed length]
A	0
B	110
C	10
D	111

→ time ↑

- Suppose the same character stream for first 10 seconds, i.e., "ABCAA"
- The corresponding source encoder output is 0 | 110 | 10 | 0 | 0 → time ↑
- The source encoder output bit rate for above case $= \frac{(1+3+2+1+1)}{10} = 0.8$ bits/sec

- Such a variable length coding as shown in the table above is useful and can be exploited for effective use of system resources if the source can be modelled "probabilistically". There is a RICH theory to handle those concerns, which is the well respected "Information Theory". However the details are extraneous to the scope of this lecture.

- Let us now discuss the functionality of CHANNEL ENCODER.
  - Here we just restrict ourself to the fixed-length coding case at the SOURCE ENCODER. That is shown in the first table above.

✱ Implementation 1

- Now consider the case where the Channel encoder maps the sequence of bits into → sequence of real numbers, -1 & +1
- In particular, Channel encoder performs the following mapping to generate its "sequence of real numbers"

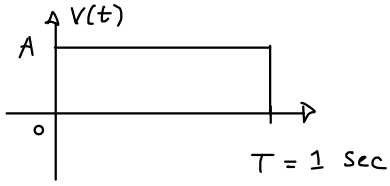
  binary "0" → real −1
  binary "1" → real +1
  } — (A)

- Next the channel encoder uses the resulting sequence of real numbers to constitute the waveform to be sent to the Tx as follows:

real -1 ⟶ $-V(t)$

real $+1$ ⟶ $+V(t)$

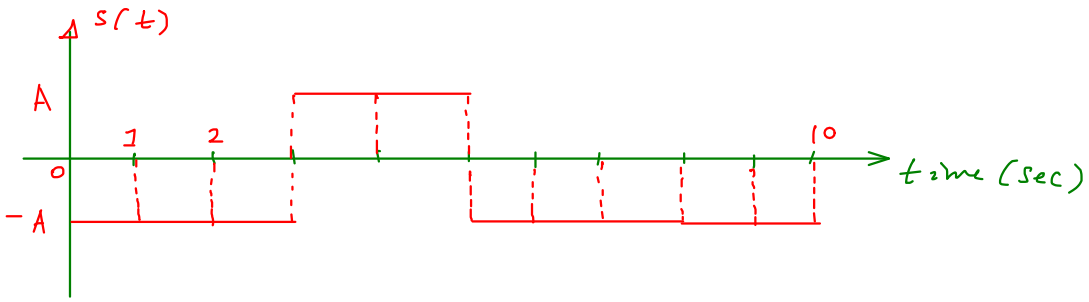
} where $V(t)$ is shown below:



Not that in the aforementioned implementation T should be equal to 1 seconds

- Recall the character stream for first 10 seconds is "ABCAA"
- The corresponding source encoder output is 00 01 10 00 00 ⟶ time

- The corresponding channel encoder waveform for first 10 seconds is as follows:

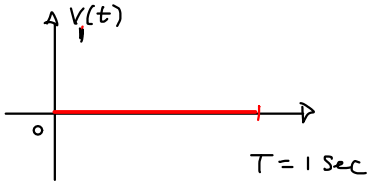


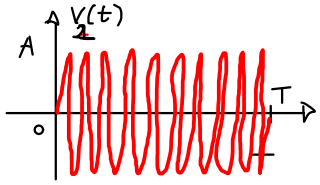
✱ Implementation 2 : Mapping from bits to real numbers is same as (A) above

real -1 ⟶ $V_1(t)$

real $+1$ ⟶ $V_2(t)$

} where $V_1(t)$ and $V_2(t)$ are shown below:

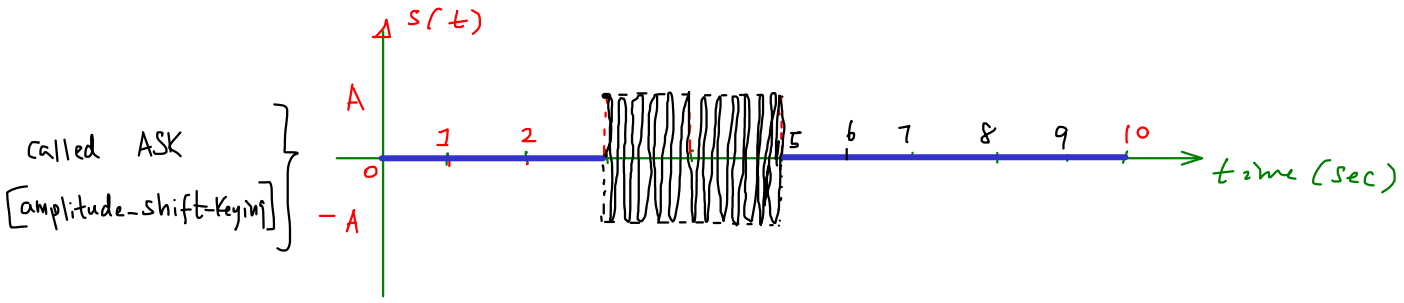




Not that in the aforementioned implementation T should be equal to 1 seconds

- Recall the character stream for first 10 seconds is "ABCAA"
- The corresponding source encoder output is 00 01 10 00 00 ⟶ time

- The corresponding channel encoder waveform for first 10 seconds is as follows:

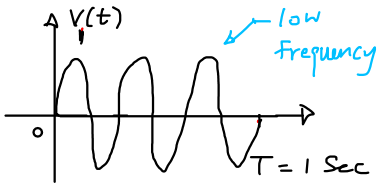


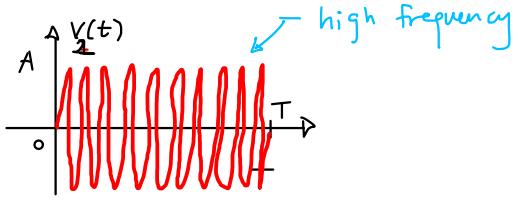
* Implementation 3: Mapping from bits to real numbers is same as (A) above

real $-1 \longrightarrow V_1(t)$

real $+1 \longrightarrow V_2(t)$

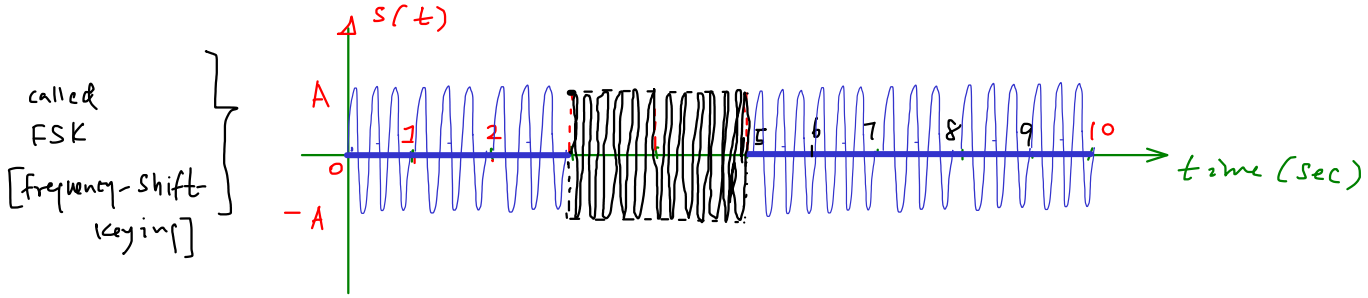
} where $V_1(t)$ and $V_2(t)$ are shown below:





Not that in the aforementioned implementation T should be equal to 1 seconds

- Recall the character stream for first 10 seconds is "ABCAA"
- The corresponding source encoder output is 00 | 01 | 10 | 00 | 00 → time↑
- The corresponding channel encoder waveform for first 10 seconds is as follows:



* Implementation 4: let us consider a different mapping of bits to real numbers

- consider the case where the Channel encoder maps the sequence of bits into a sequence of real numbers, 1, 2, 3, 4
- In particular, Channel encoder performs the following mapping to generate its "sequence of real numbers"

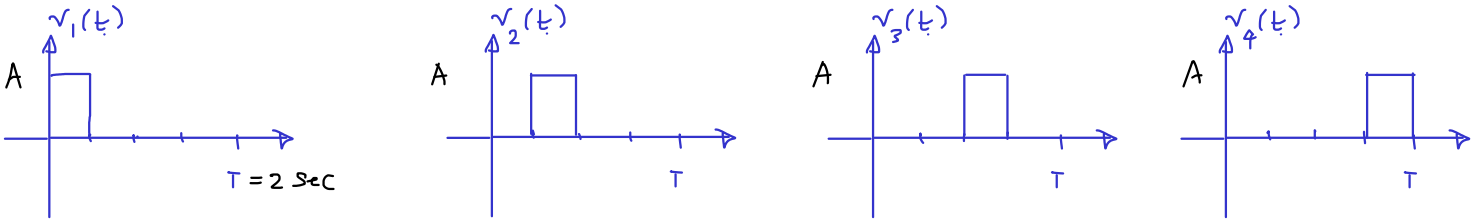
↓

"00" ⟶ 1 "10" ⟶ 3
"01" ⟶ 2 "11" ⟶ 4 } — (B)

- Next the channel encoder uses the resulting sequence of real numbers to constitute the waveform to be sent to the Tx as follows:

real 1 ⟶ $v_1(t)$
real 2 ⟶ $v_2(t)$
real 3 ⟶ $v_3(t)$
real 4 ⟶ $v_4(t)$

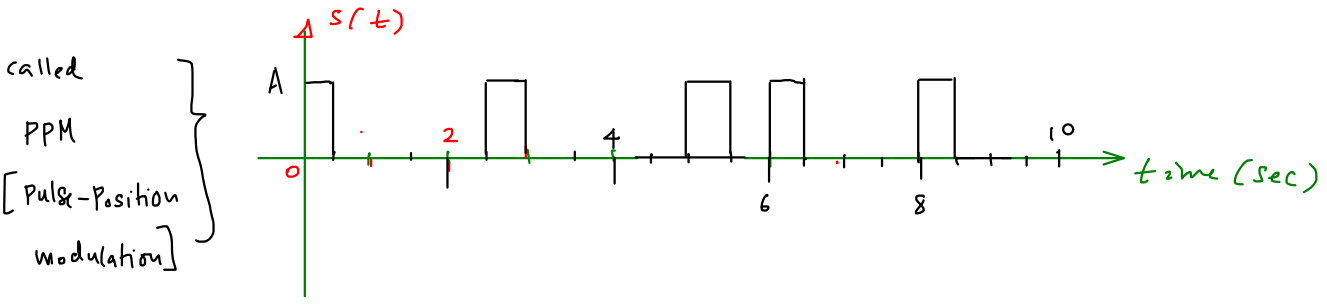
where $v_1(t)$, $v_2(t)$, $v_3(t)$ and $v_4(t)$ are shown below:



Not that in the aforementioned implementation, T should be equal to 2 seconds

- Recall the character stream for first 10 seconds is "ABCAA"
- The corresponding source encoder output is 00 | 01 | 10 | 00 | 00 ⟶ time

- The corresponding channel encoder waveform for first 10 seconds is as follows:



✱ Implementation 5 : The

Mapping from bits to real numbers is same as (B) above

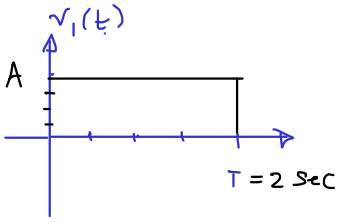
real 1 ⟶ $v_1(t)$

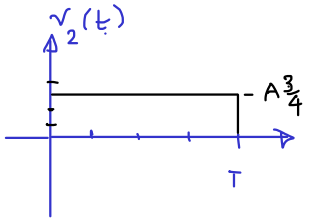
real 2 ⟶ $v_2(t)$

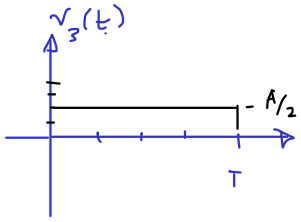
real 3 ⟶ $v_3(t)$

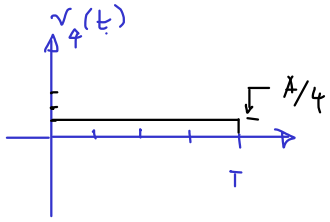
real 4 ⟶ $v_4(t)$

where $v_1(t)$, $v_2(t)$, $v_3(t)$ and $v_4(t)$ are shown below :





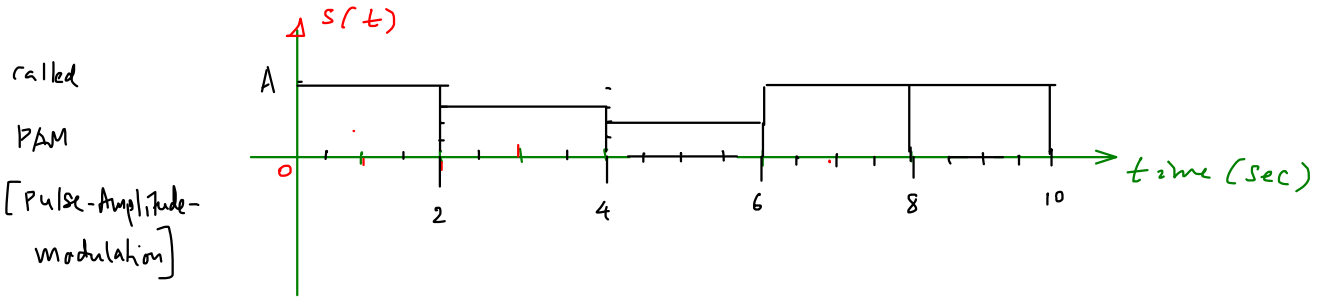




Not that in the aforementioned implementation, T should be equal to 2 seconds

- Recall the character stream for first 10 seconds is "ABCAA"
- The corresponding source encoder output is 00 | 01 | 10 | 00 | 00 ⟶ time

- The corresponding channel encoder waveform for first 10 seconds is as follows :



called PAM [Pulse-Amplitude-Modulation]

✱ As shown above the ways the source encoder and channel encoder can be configured ad nauseam ✱

✱ As a final Implementation associated with Example 1, Let us consider a mapping of bits into a "COMPLEX number sequence"

↓

"00" ⟶ $(1+j)$ "01" ⟶ $(1-j)$

"10" ⟶ $(-1+j)$ "11" ⟶ $(-1-j)$

↑ These are complex numbers

↑ These are complex numbers

Complex $(1+j)$ ⟶ $V_1(t)$

Complex $(1-j)$ ⟶ $V_2(t)$

Complex $(-1+j)$ ⟶ $V_3(t)$

Complex $(-1-j)$ ⟶ $V_4(t)$

} Each of these are of length $T = 2$ Seconds

$$V_1(t) = \text{Real}\{1+j\}\, p(t)\cos[2\pi f_c t] - \text{Imag}\{1+j\}\, p(t)\sin[2\pi f_c t]$$

$$= p(t)\cos 2\pi f_c t - p(t)\sin 2\pi f_c t$$

$$V_2(t) = p(t)\cos 2\pi f_c t + p(t)\sin 2\pi f_c t$$

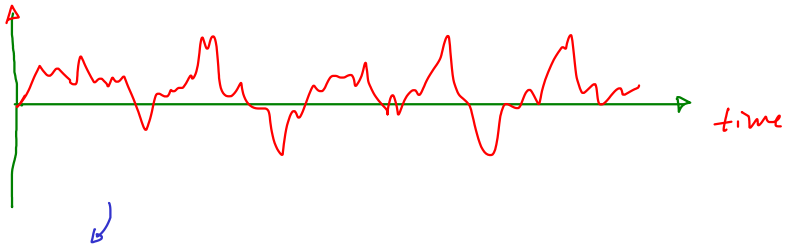
$$V_3(t) = -p(t)\cos 2\pi f_c t - p(t)\sin 2\pi f_c t$$

$$V_4(t) = -p(t)\cos 2\pi f_c t + p(t)\sin 2\pi f_c t$$

Here $p(t)$ is an appropriately chosen pulse. Details are excluded.

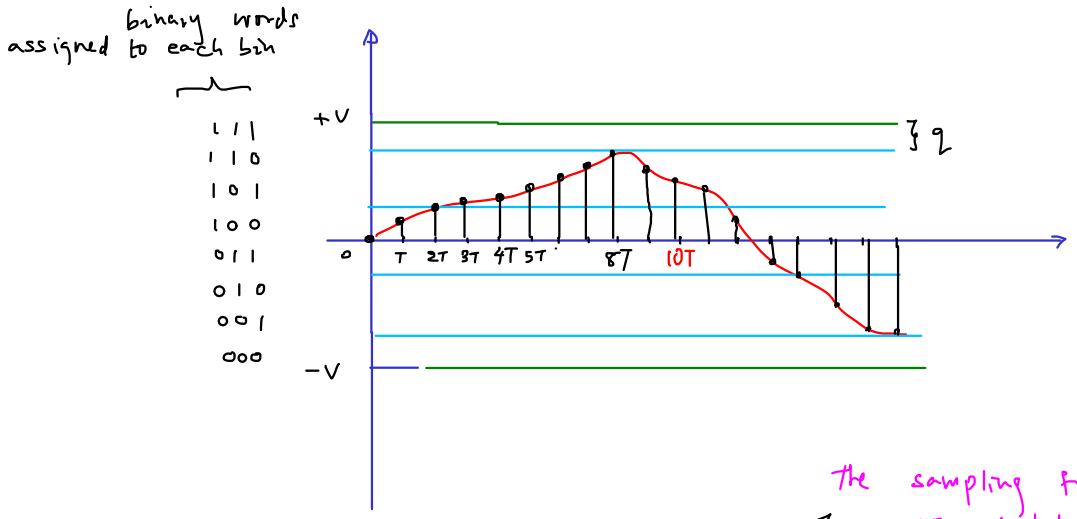
The above scheme falls under Quadrature Amplitude Modulation (QAM).

Example 2: Consider a case where the Information Source is an "analog" source, e.g., speech signal.



Let us zoom-in a small section





the red curve is the speech signal

the sampling frequency $\frac{1}{T}$ is to be decided appropriately

Recall

* First the signal is (sampled) : Those are marked in black stems above.
* Note that the Voltage range is devided into 8-bins. That is we can Lable them as shown by using 3-bit words
* Note that samples of different voltages still can fall in the same bin. For example, see the samples at t= 3T, 4T, 5T, 6T in the above figure. They all are assigned the binary word '101'.
* This means that, sampling and quantization lead to distortion at the receiver (Rx) when signal reconstruction is done.
* The aforementioned quantization distortion can not be fixed.
* However, the quantization distortion can be minimized by having MORE bins.
* For example, in the case depicted above we could have divided the voltage range into 16 bins instead of 8 bins. Of course, then the number of bits per sample is 4.

* Suppose T = 0.1 ms. Then the SOURCE Encoder output for first 1 ms would be [Ignoring the sample at t=0]

| 100 | 100 | 101 | 101 | 101 | 101 | 110 | 110 | 110 | 101 |

t=0 t=1ms

* Note that the bit rate at the end of the encoder is

$$= \frac{1}{T}\ \frac{\text{samples}}{\text{second}} \times \frac{3 \text{ bits}}{\text{sample}} = \frac{3}{T} \text{ bits/second} = \frac{3}{0.1 \times 10^{-3}} = 3 \times 10^4 \text{ bits/sec}$$

↳ i.e., 30 Kbits/Sec.

* The bottomline is that the Source encoder is generating a binary stream of data to be sent to the Channel Encoder.

* All the "Implementations" for the channel encoder that were discussed in Example 1 apply here as well.

* In addition to the aforementioned sampling mechanism, there exists other mechanisms as well. But we do not discuss those in this introductory module.